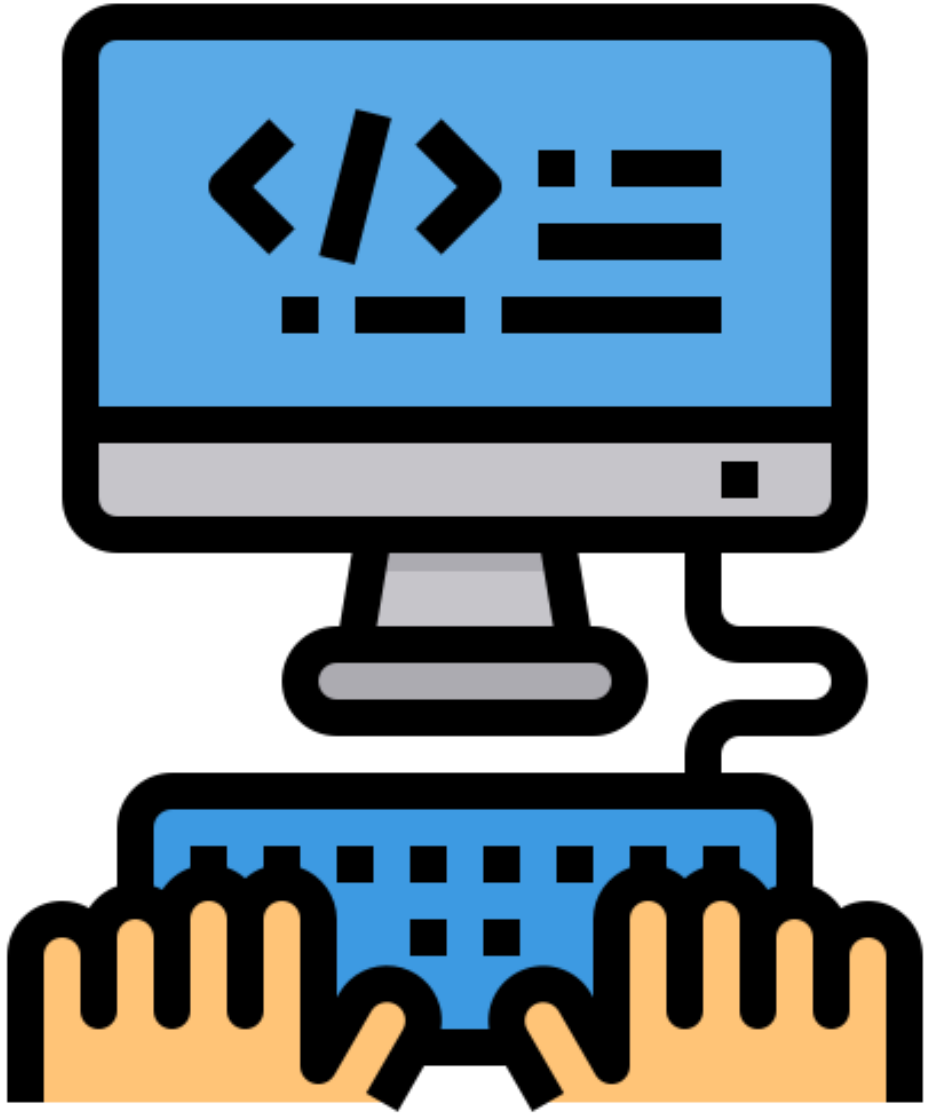
CS 24 AP Computer Science A Review

Week 7: ArrayList

DR. ERIC CHOU
IEEE SENIOR MEMBER

Topics

- What is ArrayList?
- ArrayList Methods
- ArrayList Processing
- Removal of Elements from an ArrayList



ArrayList

SECTION 1

Declaration of ArrayList

like a train. The datatype it carries is like the cargo.

- In declaring a variable of type **ArrayList** we use a statement like this:
  - **ArrayList<String> aList;**
- The word between the *angle brackets*, **<…>**, indicates the data type of the elements that the **ArrayList** will store. In this case, **aList** is declared to be an **ArrayList** each of whose elements will be a **String**.

Declaration of ArrayList

like a train. The datatype it carries is like the cargo.

- The following statement creates an **ArrayList** of **String**s and then assigns it to **aList**:

  aList = new ArrayList<String>();

- We can also declare and assign to the variable in a single statement:

  ArrayList<String> aList = new ArrayList<String>();

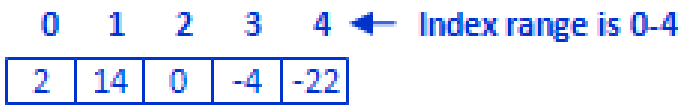
java.util.ArrayList<E>	
+ArrayList()	Creates an empty list.
+add(o: E): void	Appends a new element o at the end of this list.
+add(index: int, o: E): void	Adds a new element o at the specified index in this list.
+clear(): void	Removes all the elements from this list.
+contains(o: Object): boolean	Returns true if this list contains the element o.
+get(index: int): E	Returns the element from this list at the specified index.
+indexOf(o: Object): int	Returns the index of the first matching element in this list.
+isEmpty(): boolean	Returns true if this list contains no elements.
+lastIndexOf(o: Object): int	Returns the index of the last matching element in this list.
+remove(o: Object): boolean	Removes the first element o from this list. Returns true if an element is removed.
+size(): int	Returns the number of elements in this list.
+remove(index: int): boolean	Removes the element at the specified index. Returns true if an element is removed.
+set(index: int, o: E): E	Sets the element at the specified index.

eC Learning Channel

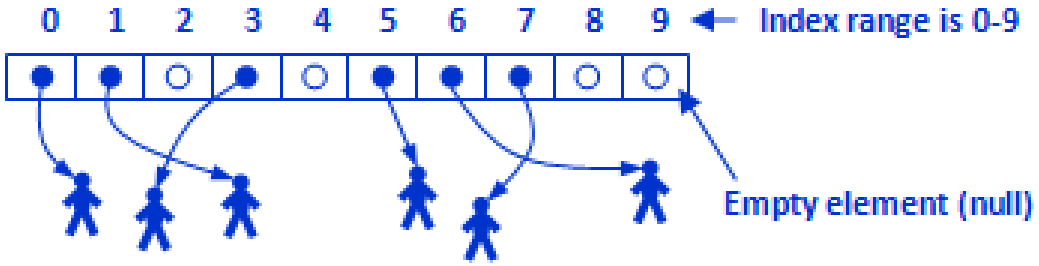
ArrayList

- An arraylist alwasys starts out empty.
- An arraylist is resizable.
- An arraylist requires an import statement.
- An arraylist can only store objects.

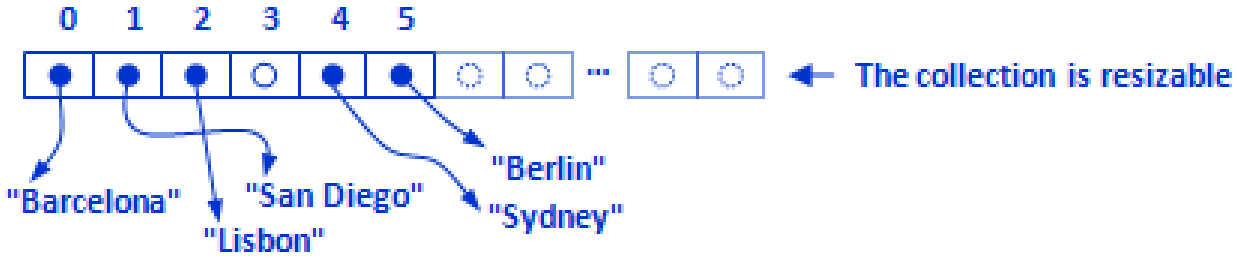
Array of 5 integers



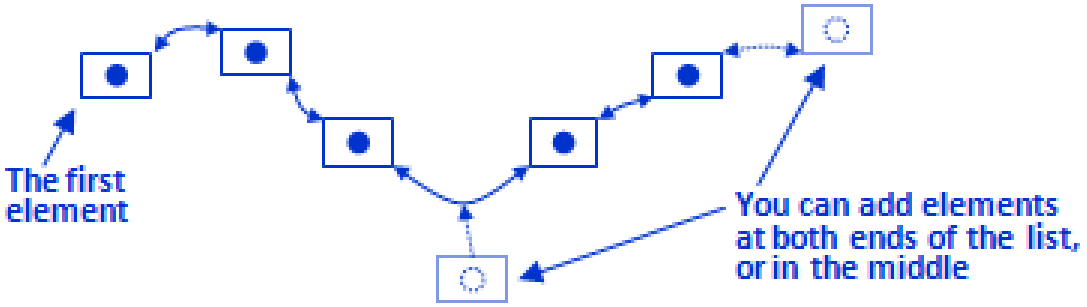
Array of 10 agents



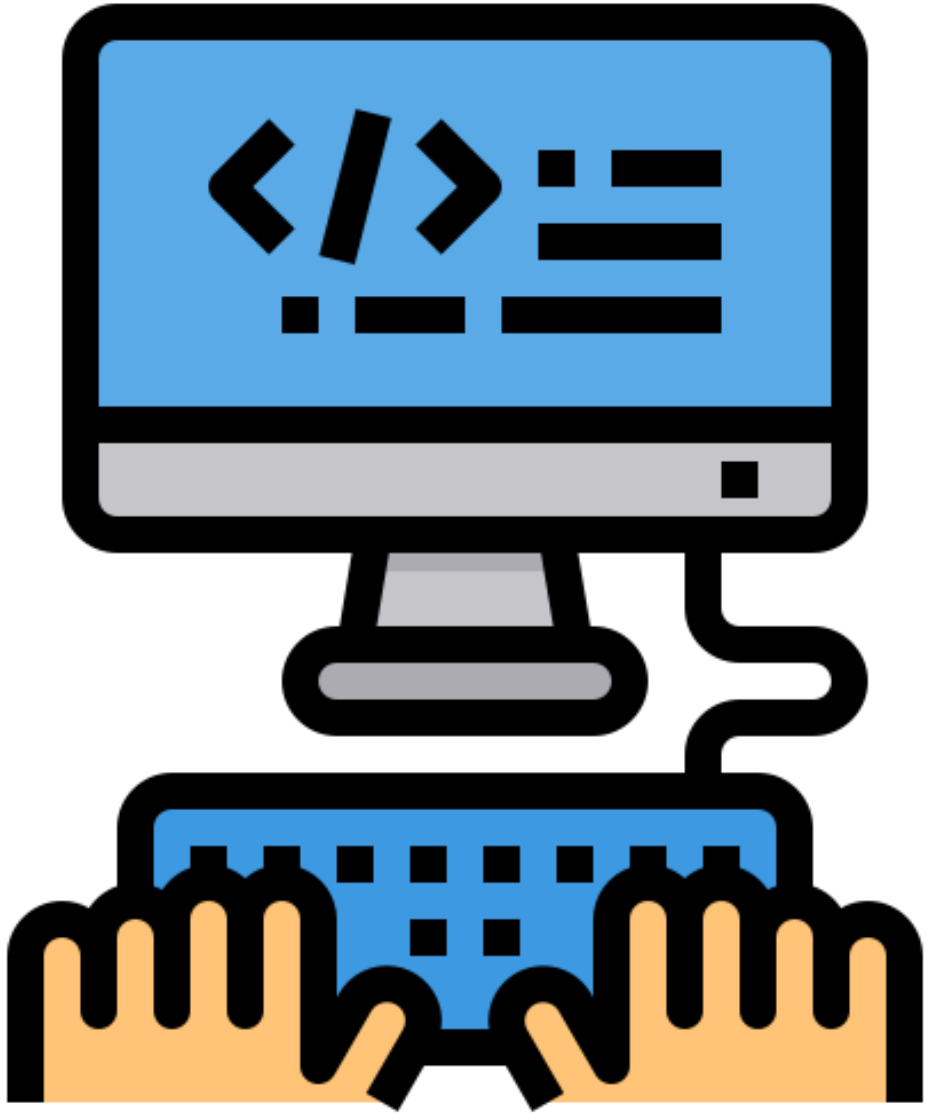
ArrayList (collection) of strings, currently contains 6 elements



LinkedList (collection)



eC Learning Channel



ArrayList Methods

SECTION 2

Differences and Similarities between Arrays and ArrayList

Operation	*Array*	*ArrayList*
Creating an array/ArrayList	`String[] a = new String[10]`	`ArrayList<String> list = new ArrayList<>();`
Accessing an element	`a[index]`	`list.get(index);`
Updating an element	`a[index] = "London";`	`list.set(index, "London");`
Returning size	`a.length`	`list.size();`
Adding a new element		`list.add("London");`
Inserting a new element		`list.add(index, "London");`
Removing an element		`list.remove(index);`
Removing an element		`list.remove(Object);`
Removing all elements		`list.clear();`

Important Methods

ArrayList<E>

- void add(E object)
- void add(int index, E Object)
- int size()
- E remove(int index)
- E get(int index)
- E set(int index, E object)

Creation of ArrayList:

Constructor, Loop Instantiation with add()

```
public static void main(String[] args){
    int count =5;
    ArrayList<Integer> aList= new ArrayList<Integer>();
    for (int i=0; i<5; i++){
        aList.add((int)(Math.random()*8));
    }
    System.out.println("Loop Creation of an ArrayList: "+aList);
}
```

Guideline

- Getter and Setter Methods using index are fine.
- Remove, add with index need extra cautions.

Selection Sort by ArrayList

- If insertion sort or selection sort is required, arraylist is a better option than array.
- It is just a finding maximum with a outer loop.

BlueJ: Terminal Window - Week3

Options

```
Before Sorting:[5, 6, 7, 3, 2, 1, 9, 10, 13, 0]
After  Sorting:[0, 1, 2, 3, 5, 6, 7, 9, 10, 13]
```

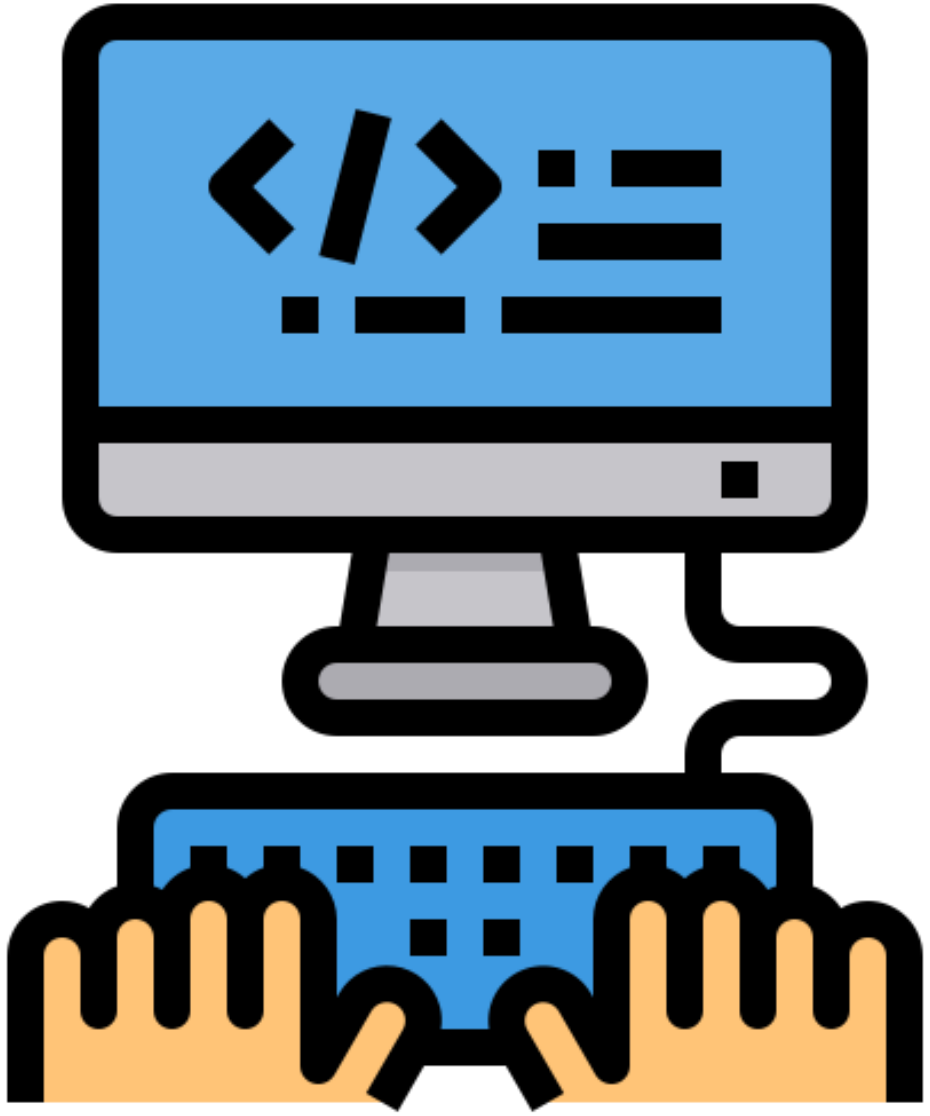
Can only enter input while your programming is runr

```java
import java.util.ArrayList;
import java.util.Arrays;

public class SelectionSort{
    public static ArrayList<Integer> selectionSort(ArrayList<Integer> alist){
        ArrayList<Integer> blist = new ArrayList<Integer>();

        while (alist.size() >0){
            int min = Integer.MAX_VALUE;
            int minIndex=0;
            for (int i=0; i<alist.size(); i++){
                if (alist.get(i)<min){
                    min = alist.get(i);
                    minIndex = i;
                }
            }
            blist.add(alist.remove(minIndex));
        }
        return blist;
    }

    public static void main(String[] args){
        ArrayList<Integer> x = new ArrayList<Integer>(
            Arrays.asList(new Integer[]{5, 6, 7, 3, 2, 1, 9, 10, 13, 0})
        );
        System.out.println("Before Sorting:"+x);
        x = selectionSort(x);
        System.out.println("After  Sorting:"+x);
    }
}
```



ArrayList Processing

SECTION 3

Reverse List 1

```
static Integer[] a = {3, 5, 8, 1, 4};
public static ArrayList<Integer> reverse(ArrayList<Integer> alist){
  int N = alist.size();
  ArrayList<Integer> blist = new ArrayList<Integer>();
  while (N>0){
      blist.add(0, alist.remove(0));
      N--;
    }
  return blist;
}
```

Reverse List 2

```
public static ArrayList<Integer> reverse2(ArrayList<Integer> alist){
   ArrayList<Integer> blist = new ArrayList<Integer>();
   for (int i=alist.size()-1; i>=0; i--){
        blist.add(alist.get(i));
     }
   return blist;
}
```

Filtering (Get Even Numbers)

```
static Integer[] a = {1, 7, 2, 4, 3, 6, 8, 5, 9};
public static ArrayList<Integer> getEven(ArrayList<Integer> alist){
    ArrayList<Integer> elist = new ArrayList<Integer>();

    for (Integer x: alist){
        if (x %2==0) elist.add(x);
    }
    return elist;
}
```

Selection Sort

```
public static ArrayList<Integer> selectionSort(ArrayList<Integer> alist){
    ArrayList<Integer> blist = new ArrayList<Integer>();

    while (alist.size() >0){
        int min = Integer.MAX_VALUE;
        int minIndex=0;
        for (int i=0; i<alist.size(); i++){
            if (alist.get(i)<min){
                min = alist.get(i);
                minIndex = i;
            }
        }
        blist.add(alist.remove(minIndex));
    }
    return blist;
}
```

Available List

```
2  import java.util.*;
3  public class AvailableList
4  {
5      public static String[] cards = {"A", "2", "7", "Q", "K"};
6
7      public static void main(String[] args){
8          ArrayList<String> availList = new ArrayList<String>(
9              Arrays.asList(cards)
10         );
11
12         // play 7
13         System.out.printf("My original cards: %s\n", availList);
14         availList.remove(new String("7"));
15         System.out.printf("My remaining cards: %s\n", availList);
16     }
17 }
```

Frequency List

```
public static void findFreq(Integer[] a){
    set.clear();
    freq.clear();
    for (int i=0; i<a.length; i++){
        int idx = set.indexOf(a[i]);
        if (idx <0){
            set.add(a[i]);
            freq.add(0);
            idx = set.size()-1;
        }
        freq.set(idx, freq.get(idx)+1);
    }
}
```

Interval List

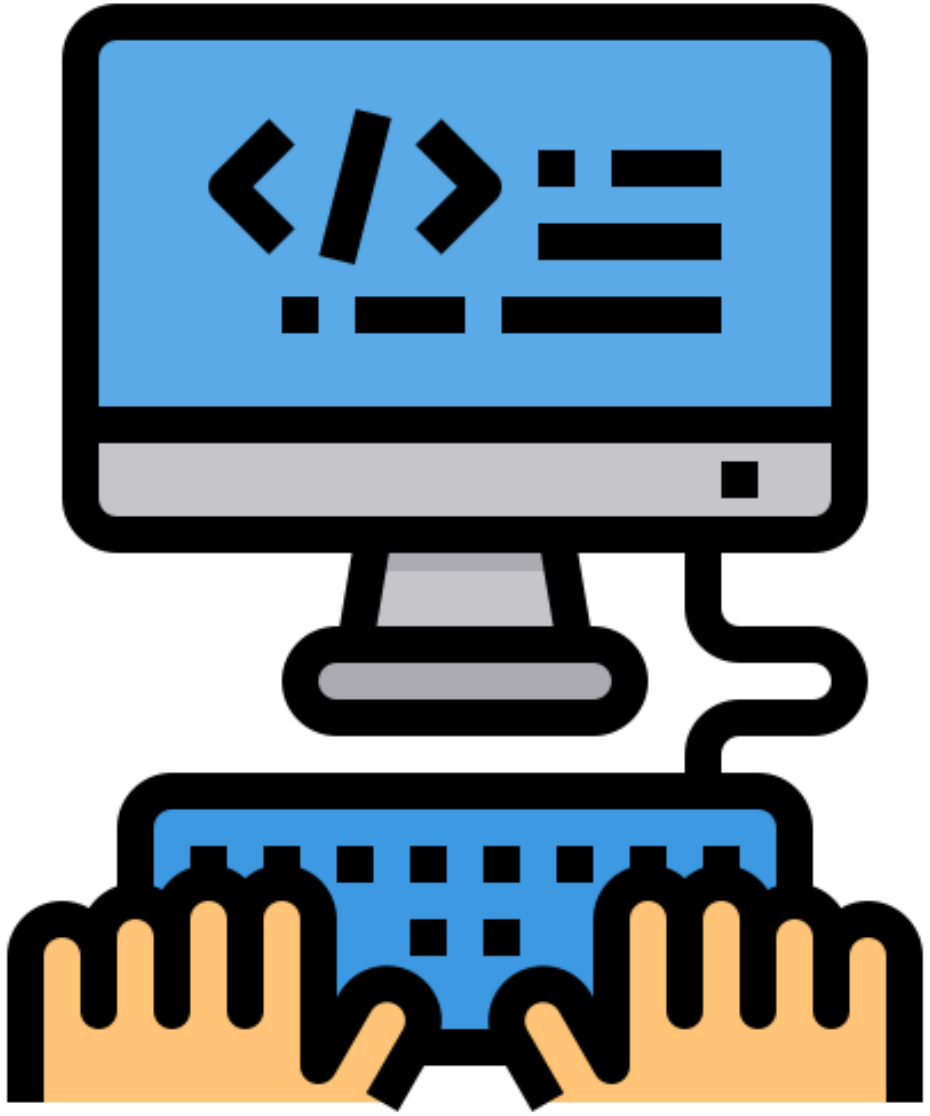
```
public static int longestRun(String data){
    ArrayList<Integer> cuts = new ArrayList<Integer>();

    cuts.add(0);
    for (int i=0; i<data.length()-1; i++){
        if (data.charAt(i) != data.charAt(i+1)) cuts.add(i+1);
     }
    cuts.add(data.length());
    System.out.println(cuts);
    int m = 0;
    int idx =-1;
    for (int i=0; i<cuts.size()-1; i++){
        int run = cuts.get(i+1)-cuts.get(i);
        if (run > m){
            m = run; idx = cuts.get(i);
          }
     }
    System.out.printf("%c has the longest run of %d\n", data.charAt(idx), m);
    return m;
}
```

Difference List

```
public static ArrayList<Integer> difference(Integer[] nike){
    ArrayList<Integer> diff = new ArrayList<Integer>();

    for (int i=0; i<nike.length-1; i++){
      Integer d = nike[i+1] - nike[i];
      diff.add(d);
    }
    return diff;
}
```

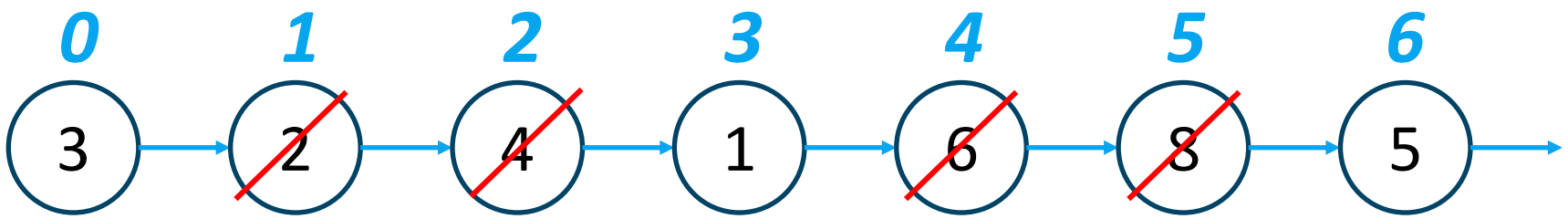


Removal of Elements from an ArrayList

SECTION 4

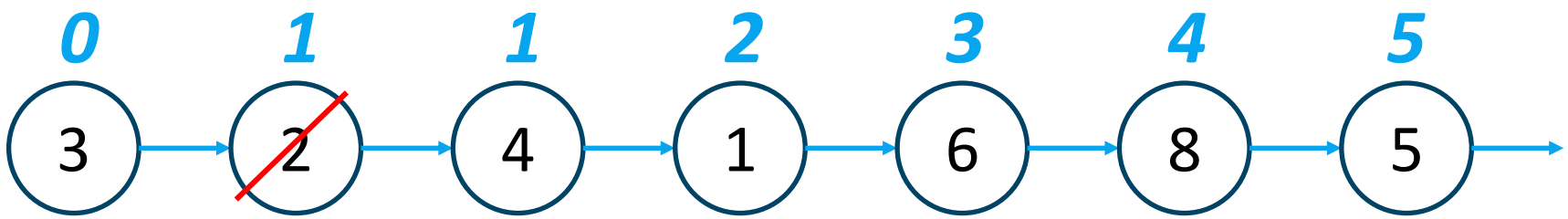
Bypass Problem (Delete Even Numbers)

Our purpose:



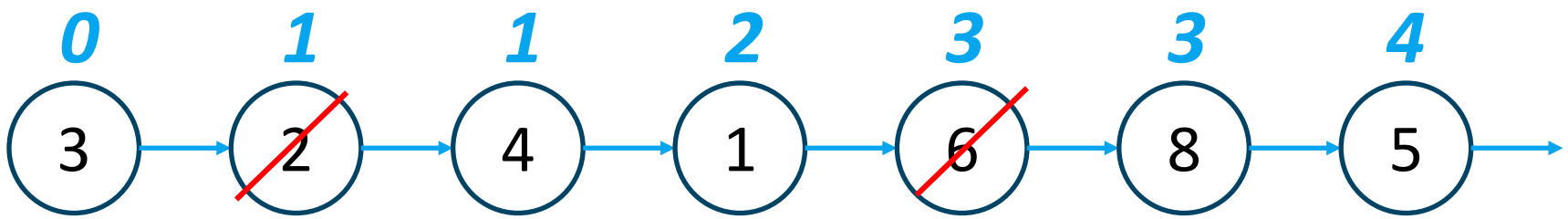
Bypass Problem (Delete Even Numbers)

After removal the index changes



Bypass Problem (Delete Even Numbers)

Missing two spots



Wrong Index

```
public static ArrayList<Integer> alist = new ArrayList<Integer>(
  Arrays.asList(new Integer[]{3, 2, 4, 1, 6, 8, 5})
);
public static void wrongList(ArrayList<Integer> alist){
  for (int i=0; i<alist.size(); i++){
      if (alist.get(i) % 2==0) alist.remove(i);
    }
}
```

```
alist before removal: [3, 2, 4, 1, 6, 8, 5]
alist after removal: [3, 4, 1, 8, 5]
```

Backward Traversal

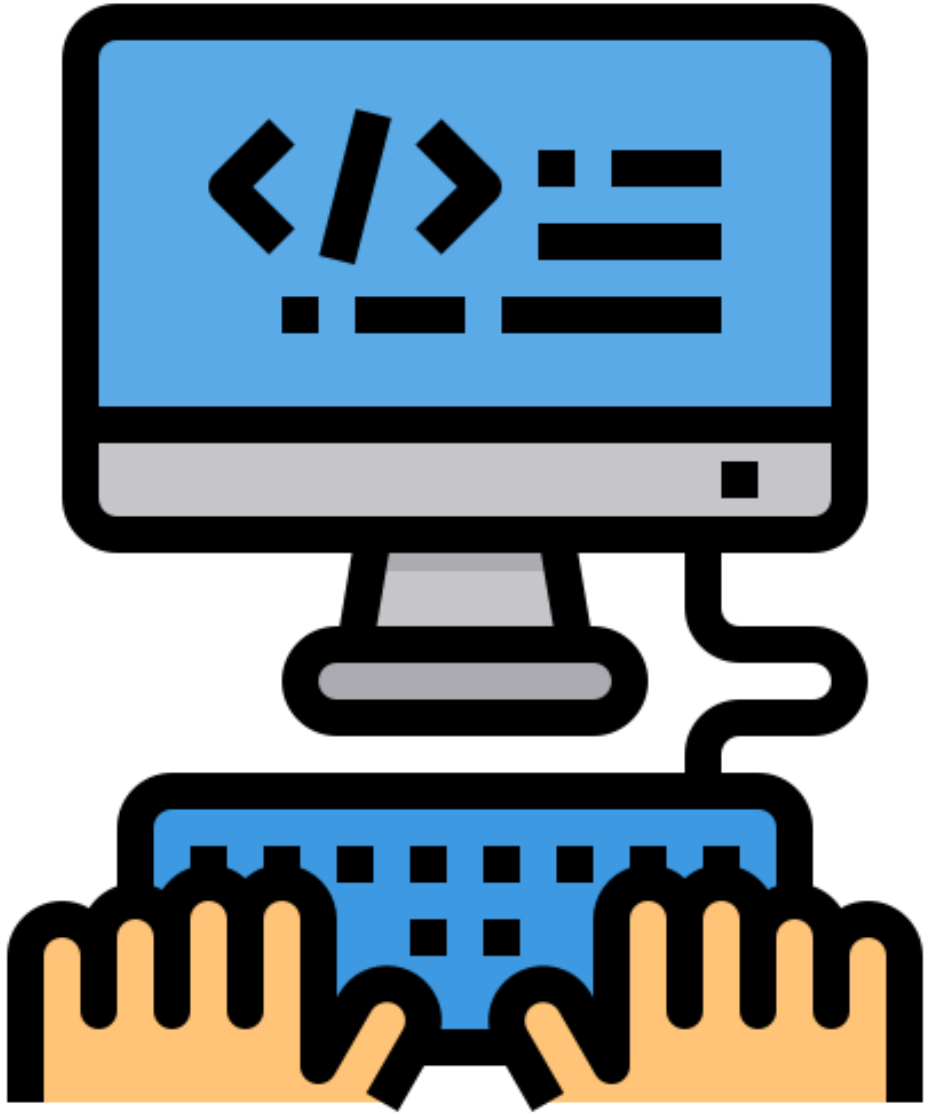
```
public static void backwardTraversal(ArrayList<Integer> alist){
  for (int i=alist.size()-1; i>=0; i--){
      if (alist.get(i)%2==0) alist.remove(i);
    }
}
```

```
alist before backward traversal: [3, 2, 4, 1, 6, 8, 5]
alist after backward traversal: [3, 1, 5]
```

Delete Without Advancing Index

```
public static void forwardTraversalWithoutAdvancing(ArrayList<Integer> alist){
    for (int i=0; i<alist.size(); ){
        if (alist.get(i)%2==0) alist.remove(i);
        else i++;
     }
}
```

```
alist before forward no advancing traversal: [3, 2, 4, 1, 6, 8, 5]
alist after forward no advancing traversal: [3, 1, 5]
```



Practice MCQ Questions

SECTION 5

Multiple Choice (1)

Consider the following code segment:

```
ArrayList list = new ArrayList();
list.add("A");
list.add("B");
list.add(0, "C");
list.add("D");
list.set(2, "E");
list.remove();
System.out.println(list);
```

What is printed as a result of executing the code segment?

Multiple Choice (1)

Ⓐ `[A, B, C, D, E]`

Ⓑ `[C, E, D]`

Ⓒ `[A, D, E]`

Ⓓ `[A, C, D, E]`

Multiple Choice (1)

Ⓐ `[A, B, C, D, E]`

Ⓑ `[C, E, D]`

Ⓒ `[A, D, E]`

Ⓓ `[A, C, D, E]`

Multiple Choice (2)

Consider the following data fields and method:

```
private ArrayList letters;
// pre-condition: letters.size() > 0
// letters contains String objects
public void letterRemover(){
   int i=0;
   while(i<letters.size()){
    if (letters.get(i).equals("A"))
       letters.remove(i);
    i++;
   }
}
```

Multiple Choice (2)

Assume that ArrayList `letters` originally contains the following `String` values:

```
[A, B, A, A, C, D, B]
```

What will be letters contain as a result of executing letterRemover()?

Multiple Choice (2)

Ⓐ `[A, B, A, A, C, D, B]`

Ⓑ `[B, C, D, B]`

Ⓒ `[B, A, C, D, B]`

Ⓓ `[A, B, A, C, D, B]`

Multiple Choice (2)

Ⓐ `[A, B, A, A, C, D, B]`

Ⓑ `[B, C, D, B]`

Ⓒ `[B, A, C, D, B]`

Ⓓ `[A, B, A, C, D, B]`

Multiple Choice (3)

Consider the following method:

```
private ArrayList myList;
// pre-condition: letters.size() > 0
// myList contains String objects
public void myMethod(){
    for (int i=0; i<myList.size()-1; i++){
      myList.remove(i);
      System.out.print(myList.get(i)+” “);
    }
}
```

Multiple Choice (3)

Assume that ArrayList `myList` originally contains the following `String` values:

```
[A, B, C, D, E]
```

What will be printed when the method above executes?

Multiple Choice (3)

Ⓐ `A B C D E`

Ⓑ `B D E`

Ⓒ `B D`

Ⓓ Nothing will be printed due to an IndexOutOfBoundsException

Multiple Choice (3)

Ⓐ `A B C D E`

Ⓑ `B D E`

Ⓒ `B D`

Ⓓ Nothing will be printed due to an IndexOutOfBoundsException

Multiple Choice (4)

Consider the following code segment:

```
ArrayList list= new ArrayList();
for (int i=0; i<=8; i++){
  list.add(new Integer(i));
}
for (int j=1; j<list.size(); j++){
  list.set(j/2, list.get(j));
}
```

What is printed as a result of executing the code segment?

Multiple Choice (4)

Ⓐ [2, 4, 6, 8, 5, 6, 7, 8]

Ⓑ [1, 2, 3, 4, 5, 6, 7, 8]

Ⓒ [1, 2, 3, 4, 1, 2, 3, 4]

Ⓓ [2, 2, 4, 4, 6, 6, 8, 8]

Multiple Choice (4)

Ⓐ [2, 4, 6, 8, 5, 6, 7, 8]

Ⓑ [1, 2, 3, 4, 5, 6, 7, 8]

Ⓒ [1, 2, 3, 4, 1, 2, 3, 4]

Ⓓ [2, 2, 4, 4, 6, 6, 8, 8]

Multiple Choice (5)

The `State` class, described as earlier, contains the method:

```
private void calculatgePercentChange(){
    // calculate and saves the percent change
    // in population from 2010 to 2020

    // missing code

    // to be completed in the Review Drill
}
```

Multiple Choice (5)

Which of the following correctly calculate the percent change in the state's population and saves the result in the state's appropriate instance variable?

Multiple Choice (5)

Ⓐ `double pctChange = (double) popChange/pop2010;`

Ⓑ `pctChange = popChange/pop2010;`

Ⓒ `pctChange = (double) popChange/pop2010;`

Ⓓ `pctChange = (double) (popChange/pop2010);`

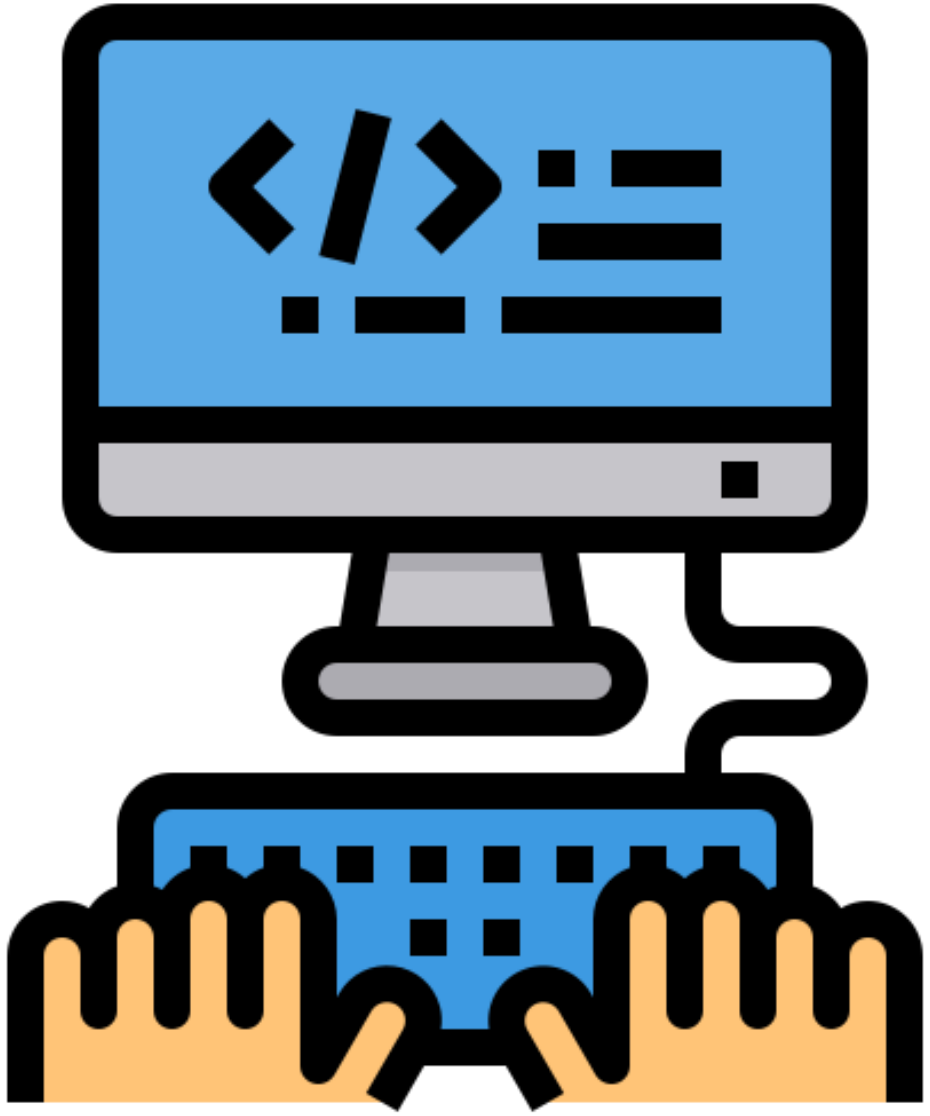
Multiple Choice (5)

Ⓐ `double pctChange = (double) popChange/pop2010;`

Ⓑ `pctChange = popChange/pop2010;`

Ⓒ `pctChange = (double) popChange/pop2010;`

Ⓓ `pctChange = (double) (popChange/pop2010);`



ArrayList Algorithms

SECTION 6

Cut List (run size)

```
public ArrayList<Integer> getCutList(String s){
    ArrayList<Integer> cuts = new ArrayList<Integer>();
    cuts.add(0);
    for (int i=1; i<s.length(); i++){
      if (s.charAt(i-1) != s.charAt(i)) cuts.add(i);
    }
    cuts.add(s.length());
    return cuts;
}
```

Prime Factor List

```
public int smallestPrimeFactor(int n){
  if (n<=2) return n;
  for (int i=2; i<=Math.sqrt(n); i++){
    if (n%i==0) return i;
  }
  return n;
}
```

Prime Factor List

```
ArrayList<Integer> factors = new ArrayList<Integer>();
ArrayList<Integer> powers = new ArrayList<Integer>();
public void getPrimeFactorList(int n){
    int x = 0;
    while (n > 1){
        x = smallestPrimeFactor(n);
        if (factors.indexOf(x) < 0){
            factors.add(x); powers.add(0);
        }
        int idx = factors.indexOf(x);
        powers.set(idx, powers.get(i)+1);
        n /= x;
    }
}
```